

Semaphores

- Semaphores are integers variables that are utilized to provide solution to the critical section problem.
- A semaphore is represented by S and apart from initialization it is utilize by two atomic operations, wait() and signal() for process synchronization.
- Wait(): The wait operation decrease the value of S, in case it is positive. On the other hand in case S is negative or zero, at that point no activity is performed.

```
wait(S)
{
    while (S<=0);
    S- - ;
}
```

- Signal(): The signal() operation increases the value of its semaphore S.

```
signal(S)
{
    S++;
}
```

- All changes to the integer value of the semaphore in the wait () and signal() function must be executed individually. That is, the point at which one process adjusts the semaphore value, no other process can at the same time change that equivalent semaphore value.
- Furthermore, on account of wait (S), the testing of the integer value of $(S \leq 0)$, with its conceivable adjustment $(S- -)$, must be executed without interruption.

Properties of Semaphores

1. It's basic and consistently have a non-negative integer value.
2. Works with numerous processes.
3. Can have a wide range of critical section with various semaphores.
4. Each critical section has exceptional access semaphores.
5. Can allow numerous processes into the critical section at the same time, if desirable.

Types of Semaphores

- There are two primary categories of semaphores for example counting semaphores and binary semaphores. Insights regarding these are given as below:
 1. **Counting Semaphores:** These are integer number value semaphores and have an unlimited value space. These semaphores are utilized to facilitate the resource access, where the semaphore count is the quantity of accessible resources. On the off chance that the resources are added, semaphore count naturally increases and if the resources are removed, the count is decremented.

2. **Binary Semaphores:** The binary semaphores resemble counting semaphores yet their value is limited to 0 and 1. The wait() function possibly works when the semaphore is 1 and the signal operation succeeds when semaphore is 0. It is sometimes simpler to execute binary semaphores than counting semaphores.

Semaphore Usage

In practice, semaphores can take on one of two structures:

- Binary semaphores can have one of the two values, 0 or 1. They can be utilized to provide solution to the critical section problem as portrayed above, and can be utilized as mutexes on systems that don't give a different mutex mechanism. The utilization of mutexes for this reason is given in figure below.

```
do {  
    waiting(mutex) ;  
  
    // critical section  
  
    signal(mutex) ;  
  
    // remainder section  
}while (TRUE) ;
```

Mutual-exclusion usage with semaphores

- Counting semaphores can take on any integer value, and are generally used to count the remaining number of some resource. The counter is defined to the value of such resources available in the system, and at point the counting semaphore is more than zero, at that point a process can enter a critical section and utilize one of the resources. When the counter gets up to zero (or negative in certain executions), then the process obstructs another process free a resource and increase the counting semaphore with a signal call.
- Semaphores can likewise be utilized to synchronize certain tasks between processes. For instance, assume it is significant that process P1 execute instruction S1 before process P2 executes instruction S2.
 - First we make a semaphore named sync that is shared by the two processes, and defined it to zero.
 - At that point in process P1 we embed the code:

```
S1;  
signal( sync );
```
 - and in process P2 we embed the code:

```
wait( sync );  
S2;
```

- Since sync was initialized to 0, process P2 will block on the wait until P1 executes the call to signal.

Advantages of Semaphores

Some of the benefits of semaphores are as per the following:

1. Semaphores permit just one process into the critical section. They follow the mutual exclusion standard carefully and are substantially more effective than some different techniques for synchronization.
2. There is no resource wastage in view of busy waiting in semaphores as processor time isn't wasted pointlessly to check if a condition is satisfied to enable a process to get to the critical section.
3. Semaphores are implemented in the machine independent code of the microkernel. So they are machine independent.

Disadvantages of Semaphores

Some of the disadvantages of semaphores are as per the following:

1. Semaphores are complicated so the wait and signal function must be designed in the right order to avoid deadlocks.
2. Semaphores are unfeasible for last scale use as their utilization prompts loss of modularity. This happens in light of the fact that the wait and signal tasks avoid the formation of an organized design for the system.
3. Semaphores may prompt a priority inversion where low priority processes may access the critical section first and high priority processes later.

Classical Problems of Synchronization

The accompanying classic problems are utilized to test for all new proposed synchronization algorithms.

1. The Bounded-Buffer Problem
2. The Reader-Writer Problem
3. Dining Philosopher Problem

4.6.1 The Bounded-Buffer Problem

- This is a generalization of the producer consumer problem wherein access is managed by a shared group of buffers of a restricted size.
- We expect that the pool comprises of n buffers, each fit for holding one item.
- The mutex semaphore gives mutual exclusion to gets to the buffer pool and is initialized to 1.
- The empty and full semaphores count the quantity of empty and full buffers. The semaphore empty is introduced to the value n; the semaphore full is initialized to the value 0.

```

do {
    ...
    // produce an item in nextp
    ...
    wait(empty);
    wait(mutex);
    ...
    // add nextp to buffer
    ...
    signal(mutex);
    signal(full);
} while (TRUE);

```

The structure of the producer process.

```

do {
    wait (full);
    wait (mutex);
    ...
    // remove an item from buffer to nextc
    ...
    signal(mutex);
    signal(empty);
    ...
    // consume the item in nextc
    ...
} while (TRUE);

```

The structure of the consumer process.

The Reader-Writer Problem

- In the readers-writers issue there are a few processes (named readers) who just read the shared data, and never attempt to change it, and there are different processes (named writers) who may update the data with reading it. There is no restriction to what number of readers can access the data at the same time, however when a writer access the data, it needs to be exclusive access i.e. no other process will write data.
- There are a few variations to the readers-writer problem, most based on relative needs of readers versus writers.
 - The first readers-writers problem offers priority to readers. In this problem, if a reader needs access to the data, and there isn't now a writer accessing it, at that point access is allowed to the reader. An answer for this problem can prompt starvation of the writers, as there could generally be more readers coming along to access the data. (A constant flow of readers will come in front of waiting writer so far as there is as of now effectively another reader access the data, on that grounds that the author is compelled to wait until the data is idle, which may never occur if there are sufficient readers.)

- The second readers-writers problem offers priority to the writers. In this issue, when a writer needs access to the data it comes to the header of the queue - All waiting readers are blocked, and the writer gets access to the data when it becomes accessible. In this arrangement the readers might be starved by a constant flow of writers.
- The accompanying code is a case of the first readers-writers problem, and includes a significant counter and two binary semaphores:
 - readcount is utilized by the reader processes, to check the quantity of readers at present access the data.
 - mutex is a semaphore utilized uniquely by the readers for controlled access to readcount.
 - rw_mutex is a semaphore used to block and release the writers. The first reader to get to the data will set this lock and the last reader to leave will release it; The rest of the readers don't contact rw_mutex.
 - Note that the main peruser to go along will hinder on rw_mutex if there is at present an essayist getting to the data, and that every single after peruser will just obstruct on mutex for their go to increase readcount.

```

do {
    wait(wrt);
    ....
    // writing is performed
    ....
    signal(wrt);
} while (TRUE);

```

The structure of a writer process.

```

do {
    wait (mutex);
    readcount++;
    if (readcount 1)
        wait (wrt);
    signal(mutex);
    ...
    // reading is performed
    ...
    wait(mutex);
    readcount--;
    if (readcount 0)
        signal(wrt);
    signal(mutex);
} while (TRUE);

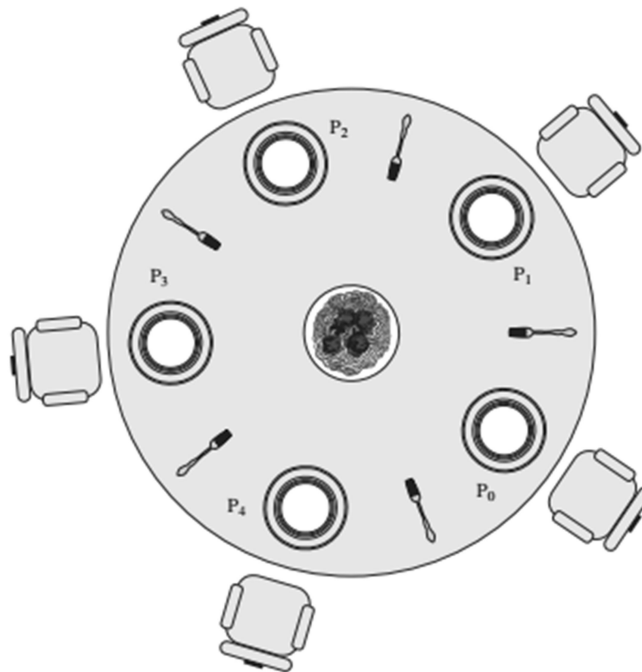
```

The structure of a reader process.

- Some hardware executions give explicit reader-writer locks, which are accessed utilizing a contention indicating whether access is mentioned for reading or writing. The utilization of reader-writer locks is valuable for circumstance in which: (1) processes can be effectively distinguished as either readers or writers, and (2) there are fundamentally a larger number of readers than writers, making the extra overhead of the reader-writer lock pay off as far as expanded simultaneousness of the readers.

Dining Philosopher Problem

- The dining philosopher's problem is a classic synchronization problem including the distribution of restricted resources among a grouping of processes in a deadlock free and starvation-free way:
- Consider five philosophers sitting around a table, where there are five chopsticks equally distributed and a perpetual bowl of rice in the center, as appeared in the figure underneath. (There is actually one chopstick between each pair of dining philosopher).



- These philosophers spend their lives shifting back and forth between two functions: eating and thinking.
- When it is time for a philosopher to eat, it should initially obtain two chopsticks - one from their left and one from their right.
- When a philosopher goes in thinking state, it puts down the two chopsticks in their actual locations.
- One conceivable arrangement, as appeared in the accompanying code area, is to utilize a set of five semaphores (chopsticks[5]), and to have each hungry philosopher

looking out for their first left chopstick (`chopsticks[i]`), and after that look out for their second chopstick (`chopsticks[(i + 1) % 5]`).

- In any case, assume that every one of the five philosopher get hungry simultaneously, and everyone begin by grabbing their left chopstick. They at that point search for their right chopstick, but since it is inaccessible, they wait for it, perpetually, and in the end every one of the philosopher starve because of the subsequent deadlock.

```
do {  
    wait(chopstick[i]);  
    wait(chopstick[(i+1) % 5]);  
    ...  
    // eat  
    ...  
    signal(chopstick[i]);  
    signal(chopstick[(i+1) % 5]);  
    ...  
    // think  
    ...  
} while (TRUE);
```

The structure of philosopher i.

- Some potential answers for the issue include:
 - Just enable four philosophers to dine simultaneously.
 - Enable thinkers to get chopsticks just when both are available, in a critical section. (All or nothing assignment of basic resources.)
 - Utilize an asymmetric arrangement, in which odd philosophers get their left chopstick first and even philosophers get their right chopstick first. (Will this arrangement consistently work? Imagine a scenario where there are an even number of philosophers.)
- Note cautiously that a deadlock free answer for the eating philosophers issue doesn't really ensure a without starvation one. (While a few or even the majority of the philosophers might have the option to continue ahead with their typical existences of eating and thinking, there might be one unfortunate soul who never is by all accounts ready to get the two chopsticks simultaneously).